

# RR7x Capital Hub — Release Técnico e Estratégico

---

**Versão:** 2.0

**Data:** Maio de 2026

**Classificação:** Documento Mestre do Sistema

**Plataforma:** rr7x-portal.vercel.app

---

## Sumário

---

- [1. Visão Geral da Plataforma](#)
  - [2. Arquitetura Técnica](#)
  - [3. Estrutura de Banco de Dados](#)
  - [4. Hierarquia de Acessos e Permissões](#)
  - [5. Gestão de Escritórios e Usuários](#)
  - [6. Fluxos Operacionais do Sistema](#)
  - [7. Pipeline de Inteligência — Deal Analysis](#)
  - [8. Integrações Externas e Dados de Mercado](#)
  - [9. Modelo de Negócio e Planos de Assinatura](#)
  - [10. Segurança, Autenticação e Autorização](#)
  - [11. Proteção de Dados e Criptografia](#)
  - [12. Auditoria, Rastreabilidade e Conformidade](#)
  - [13. Estratégias Anti-Fraude e Prevenção de Vazamentos](#)
  - [14. LGPD e Privacidade](#)
  - [15. Escalabilidade, Performance e Redundância](#)
  - [16. Cenários de Uso Reais](#)
  - [17. Limitações Estruturais e Proteções do Sistema](#)
  - [18. Visão Estratégica — Produto SaaS Escalável](#)
- 

## 1. Visão Geral da Plataforma

---

### 1.1 O que é o RR7x Capital Hub

O RR7x Capital Hub é uma plataforma SaaS B2B de inteligência para operações de M&A, estruturação de ativos e originação de deals no mercado financeiro brasileiro. A plataforma combina automação via inteligência artificial, gestão multiempresa (multiescritório) e conformidade regulatória em uma única interface, permitindo que escritórios de assessoria financeira produzam análises institucionais completas em fração do tempo convencional.

O sistema não é um chatbot nem uma ferramenta genérica de IA. É um pipeline estruturado de agentes especializados — cada um com papel e escopo definidos — que produz relatórios técnicos prontos para uso em processos de due diligence, comitê de investimento e captação.

1.2 Público-alvo

- **Escritórios de assessoria financeira** registrados na CVM (assessores autônomos, plataformas de investimento, boutiques de M&A)
- **Gestores de fundos** que originam operações estruturadas
- **Investidores institucionais** que recebem análises via compartilhamento externo
- **Operadores de mercado de capitais** que necessitam de estruturação de CRI, CRA, debêntures e outros instrumentos

1.3 Proposta de Valor Central

| Sem a plataforma                                      | Com a plataforma                                             |
|-------------------------------------------------------|--------------------------------------------------------------|
| Análise de deal: 5–15 dias úteis                      | Análise completa: 30–60 minutos                              |
| Custo por analista sênior/deal: R\$ 15.000–R\$ 50.000 | Custo por análise: R\$ 2.500–R\$ 5.000 (avulso) ou ilimitado |
| Rastreabilidade: manual, em planilhas                 | Rastreabilidade: automática, com audit log regulatório       |
| Conformidade ICVM 598: manual                         | Conformidade ICVM 598: atestação digital integrada           |
| Relatórios: formato variável por analista             | Relatórios: padronizados, institucionais, exportáveis        |

2. Arquitetura Técnica

2.1 Stack Principal

|                            |                                                          |
|----------------------------|----------------------------------------------------------|
| Camada de Apresentação:    | Next.js 16.2.4 (App Router) + React 19                   |
| Estilização:               | Tailwind CSS v4 (sistema de cores oklch)                 |
| Tipagem:                   | TypeScript (strict mode)                                 |
| Autenticação:              | Supabase Auth (JWT + SSR cookies)                        |
| Banco de Dados:            | Supabase (PostgreSQL 15 gerenciado)                      |
| Segurança no Banco:        | Row Level Security (RLS) em todas as tabelas             |
| Motor de IA:               | Anthropic Claude Sonnet (claude-sonnet-4-6)              |
| Pagamentos:                | Stripe (subscriptions + one-time payments)               |
| Email transacional:        | Resend                                                   |
| Armazenamento de arquivos: | Supabase Storage (buckets privados + público para logos) |
| Hospedagem / CDN:          | Vercel (Edge Network global)                             |
| Processamento de docs:     | Mammoth (DOCX), pdf-parse (PDF), xlsx (planilhas)        |

Exportação: @react-pdf/renderer (PDF), pptxgenjs (PPTX), xlsx (Excel)

## 2.2 Princípios Arquiteturais

**Server-first.** Toda lógica sensível roda em Server Components ou Route Handlers no servidor. O navegador nunca recebe chaves de API, dados de outros usuários ou tokens internos.

**Zero trust no cliente.** Cada requisição à API valida sessão independentemente — o servidor nunca confia no estado que o cliente afirma ter. Um token expirado ou inválido retorna 401 mesmo que o middleware não tenha capturado.

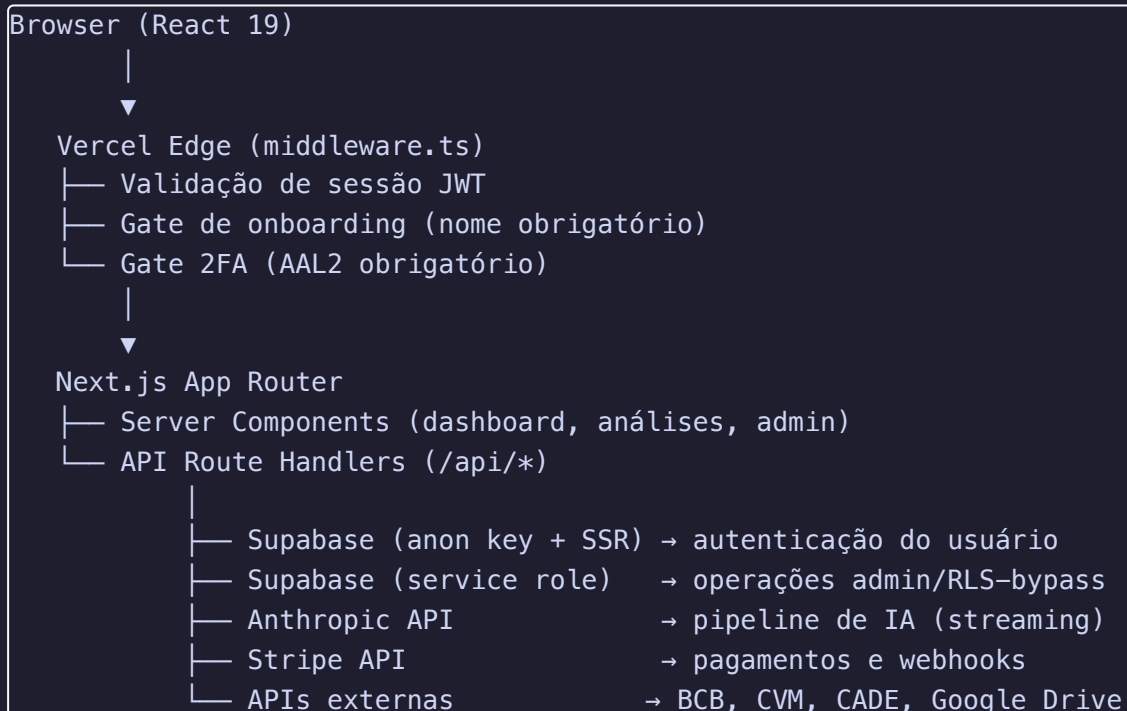
**Dois clientes Supabase distintos.** O sistema mantém dois clientes separados em toda a camada de API:

- `supabase-server` (cliente SSR com cookies do usuário): respeita RLS, só acessa dados do usuário autenticado
- `createAdminClient` (service role): bypassa RLS para operações administrativas legítimas — usado exclusivamente em Route Handlers protegidos por verificação de role

**Streaming de IA.** As análises são transmitidas token a token via `ReadableStream` HTTP. O frontend começa a exibir o conteúdo assim que os primeiros tokens chegam, sem esperar o output completo.

**Idempotência no Stripe.** O webhook de pagamento registra o `stripe_event_id` antes de processar, prevenindo que o mesmo evento seja processado duas vezes em caso de retry do Stripe.

## 2.3 Diagrama de Camadas



## 2.4 Estrutura de Diretórios

```
/
├── app/
│   ├── auth/          → login, cadastro, 2FA, definir senha, completar
│   ├── cadastro
│   ├── dashboard/
│   │   ├── (main)/    → área principal do usuário (layout com Sidebar)
│   │   ├── analise/    → listagem e detalhe de análises
│   │   ├── nova-analise/ → formulário de intake de deal
│   │   ├── escritorio/ → perfil do escritório
│   │   ├── equipe/     → gestão de membros do escritório
│   │   ├── aprendizados/ → feedbacks específicos do escritório
│   │   ├── planos/     → assinaturas e faturamento
│   │   ├── conta/      → configurações pessoais
│   │   ├── admin/      → painel administrativo (role: admin)
│   │   └── nova-analise/ → rota alternativa de intake
│   └── api/
│       ├── analise/    → CRUD de análises + pipeline de IA
│       ├── analise-status/ → status e controle do pipeline
│       ├── admin/      → gestão de usuários, escritórios, planos, agentes
│       ├── auth/       → callbacks de autenticação
│       ├── checkout/   → criação de sessão Stripe
│       ├── escritorio/ → perfil do escritório (self-service)
│       ├── gerente/    → convite de membros (role: gerente)
│       ├── upload-logo/ → upload de logo do escritório
│       ├── upload-url/ → geração de URLs assinadas para documentos
│       ├── validar-drive/ → validação de acessibilidade do Google Drive
│       └── webhook/    → processamento de eventos Stripe
├── lib/
│   ├── anthropic.ts    → cliente Anthropic + definição do modelo
│   ├── audit.ts        → sistema de audit log
│   ├── bcb-data.ts     → indicadores do Banco Central (SELIC, IPCA, USD,
│   ├── cade-check.ts   → verificação de notificação ao CADE
│   ├── crypto.ts       → criptografia AES-256-GCM para PII
│   ├── cvm-data.ts     → dados abertos CVM (debêntures, CRI, CRA, cias
│   ├── doc-reader.ts   → leitura de documentos (PDF, DOCX, XLSX, etc.)
│   ├── drive.ts        → validação de acesso a Google Drive
│   ├── email.ts        → emails transacionais (Resend)
│   ├── excel-export.ts → exportação para Excel
│   ├── financial-model-excel.ts → modelo financeiro em Excel
│   ├── get-role.ts     → resolução de role e contexto do usuário
│   ├── pdf-document.tsx → geração de PDF via React
│   ├── pptx-export.ts  → exportação para PowerPoint
│   ├── rate-limit.ts   → controle de taxa via Supabase RPC
│   └── schemas.ts      → validação de inputs com Zod
```

|                         |                                          |
|-------------------------|------------------------------------------|
| — share-token.ts        | → tokens de compartilhamento HMAC-SHA256 |
| — stripe.ts             | → cliente Stripe + definição de planos   |
| — supabase-server.ts    | → clientes Supabase (SSR + admin)        |
| — types.ts              | → tipagem TypeScript global              |
| — components/           |                                          |
| — AdminNav.tsx          | → navegação do painel admin              |
| — AgentMark.tsx         | → marcador de output de agente           |
| — AgentRow.tsx          | → linha de agente no pipeline            |
| — DRSBar.tsx            | → barra de progresso DRS                 |
| — DealPipelinePanel.tsx | → painel visual do pipeline              |
| — LiveLog.tsx           | → log em tempo real do streaming         |
| — PhaseLabel.tsx        | → rótulo de fase do pipeline             |
| — PipelineCard.tsx      | → card de deal no pipeline               |
| — Sidebar.tsx           | → navegação lateral principal            |
| — Topbar.tsx            | → barra superior                         |

## 3. Estrutura de Banco de Dados

### 3.1 Visão Conceitual

O banco é PostgreSQL gerenciado pelo Supabase, com Row Level Security (RLS) ativo em todas as tabelas. Nenhuma tabela aceita acesso anônimo direto — cada política define com precisão quem pode ler, inserir, atualizar ou deletar cada registro.

### 3.2 Tabelas Principais

#### **auth.users** — Supabase Auth (gerenciada internamente)

Tabela nativa do Supabase. Armazena credenciais, email, metadados do usuário

( **user\_metadata.nome** ), status de banimento ( **banned\_until** ) e informações de MFA. Nunca é manipulada diretamente via SQL — apenas via **supabase.auth.admin.\***.

#### **public.perfis**

|               |             |                                       |
|---------------|-------------|---------------------------------------|
| user_id       | UUID PK     | → referência para auth.users          |
| nome          | TEXT        | → nome do usuário                     |
| role          | TEXT        | → 'admin'   'gerente'   'assessor'    |
| escritorio_id | UUID        | → FK para escritorios (pode ser NULL) |
| criado_em     | TIMESTAMPTZ |                                       |

Tabela central de identidade de negócio. Todo usuário tem exatamente um perfil. O **role** aqui é a fonte de verdade para autorização — não existe no JWT, é consultado a cada request que requer verificação de

permissão.

Política RLS: cada usuário lê e edita apenas o próprio perfil. Leitura pública habilitada (para exibir nome/escritório de membros de um deal). Service role acessa tudo.

### public.escritorios

```
id          UUID PK
user_id     UUID      → proprietário original (fallback para admin)
nome        TEXT
cnpj        TEXT
endereco    TEXT
cidade_uf   TEXT
telefone    TEXT
email_contato TEXT
site        TEXT
tagline     TEXT
logo_url    TEXT
plano       TEXT      → plano do escritório (avulso, recorrente,
enterprise)
plano_status TEXT      → ativo | cancelado | pendente
plano_limite_analises INTEGER → NULL = ilimitado
criado_em   TIMESTAMPTZ
atualizado_em TIMESTAMPTZ
```

Representa o escritório ou assessoria como entidade jurídica e operacional. Os campos do escritório são injetados no contexto de cada análise gerada — os relatórios saem brandados com o nome, logo e dados do escritório, não com a marca RR7x.

Política RLS: usuário acessa apenas o escritório ao qual está vinculado via `perfis.escriptorio_id`.

### public.subscriptions

```
id          UUID PK
user_id     UUID → FK auth.users
stripe_customer_id TEXT
stripe_subscription_id TEXT
plano       TEXT → 'avulso' | 'recorrente' | 'enterprise'
analises_restantes INTEGER → NULL = ilimitado
status      TEXT → 'ativo' | 'cancelado' | 'pendente'
criado_em   TIMESTAMPTZ
atualizado_em TIMESTAMPTZ
```

Controla o acesso às análises. Antes de criar qualquer análise, o sistema verifica se existe uma `subscription` com `status = 'ativo'`. Se `analises_restantes` for NULL, o plano é ilimitado. Se for um número, é decrementado a cada análise criada.

Política RLS: usuário vê apenas a própria assinatura.

### public.analises

```
id          UUID PK
user_id     UUID   → proprietário da análise
nome_ativo  TEXT    → nome do ativo analisado
deal_intake JSONB   → todos os dados de intake (campos PII criptografados
em AES-256-GCM)
status      TEXT    → 'processando' | 'concluido' | 'erro'
outputs     JSONB   → outputs dos agentes por chave (ex: {"pesquisa":
"...", "diagnostico": "..."})
pdf_url     TEXT    → URL do PDF gerado (quando existente)
pipeline_stage TEXT → 'originacao' | 'analise' | 'compliance' | 'comite' |
'aprovado' | 'rejeitado'
criado_em   TIMESTAMPTZ
atualizado_em TIMESTAMPTZ
```

Tabela central do produto. Cada análise é um workspace completo: contém o intake bruto, todos os outputs gerados pelos agentes de IA, o estágio no pipeline e o status de processamento.

Política RLS: cada usuário acessa apenas as próprias análises. O admin (service role) acessa todas.

### public.deal\_members

```
id          UUID PK
analise_id  UUID → FK analyses
user_id     UUID → FK auth.users
role       TEXT → 'originador' | 'analista' | 'compliance' | 'parceiro' |
'gestor'
adicionado_em TIMESTAMPTZ
UNIQUE(analise_id, user_id)
```

Sistema de colaboração por deal. O owner de uma análise pode adicionar outros usuários como membros com roles específicos. Membros têm acesso de leitura à análise e podem participar do pipeline de aprovação.

Política RLS: membro vê apenas os próprios registros. Owner da análise pode gerenciar todos os membros do deal.

### public.deal\_pipeline\_events

```
id          UUID PK
analise_id  UUID → FK analyses
user_id     UUID → FK auth.users
user_email  TEXT
tipo       TEXT → 'stage_change' | 'comment' | 'aprovacao' | 'rejeicao'
```

```
stage_de    TEXT
stage_para  TEXT
comentario  TEXT
criado_em   TIMESTAMPTZ
```

Histórico imutável de eventos do pipeline. Cada avanço de estágio, aprovação, rejeição ou comentário é registrado com timestamp e identidade do autor. Garante rastreabilidade completa do processo de decisão de cada deal.

Política RLS: owner e membros da análise podem ver e inserir eventos.

### **public.deal\_output\_versions**

```
id          UUID PK
analise_id  UUID
step_key    TEXT → identificador do agente (ex: 'diagnostico')
version_num INTEGER → número sequencial, incrementado a cada reprocessamento
content     TEXT → output completo do agente nesta versão
criado_em   TIMESTAMPTZ
UNIQUE(analise_id, step_key, version_num)
```

Versionamento automático de outputs. Cada vez que um step é reprocessado, o output anterior é preservado. O assessor pode comparar versões e decidir qual utilizar.

### **public.deal\_step\_audit\_logs**

```
id          UUID PK
analise_id  UUID
step_key    TEXT
model_id    TEXT → ex: 'claude-sonnet-4-6'
input_tokens INTEGER
output_tokens INTEGER
intake_snapshot JSONB → snapshot exato do intake enviado para a IA
context_steps TEXT[] → quais outputs anteriores estavam no contexto
external_data JSONB → quais dados externos foram injetados (BCB, CVM, etc.)
ran_at      TIMESTAMPTZ
duration_ms  INTEGER
```

Audit log de cada execução de IA. Registra exatamente o que o modelo recebeu como input, quais dados externos foram injetados, quantos tokens foram consumidos e quanto tempo levou. Fundamental para conformidade regulatória (ICVM 598/2018) e para rastrear o raciocínio da IA em disputas futuras.

### **public.deal\_attestations**



```

id          UUID PK
analise_id  UUID
step_key    TEXT
version_num INTEGER
attested_by UUID → FK auth.users
attested_email TEXT
statement   TEXT → declaração legal assinada
ip_address  TEXT → IP do atestante no momento da assinatura
attested_at TIMESTAMPTZ
UNIQUE(analise_id, step_key, version_num)

```

Atestação digital de supervisão humana. O assessor assina digitalmente cada relatório de IA, declarando que leu, revisou e assume responsabilidade pelo conteúdo. A declaração padrão é:

*"Declaro que li, revisei e assumo responsabilidade pelo conteúdo deste relatório gerado com auxílio de IA, conforme ICVM 598/2018 e Código ANBIMA para M&A."*

O IP do atestante é capturado para prova forense.

### public.admin\_feedbacks

```

id          UUID PK
texto       TEXT → instrução ou aprendizado
ativo       BOOLEAN → se deve ser injetado nas análises
criado_em   TIMESTAMPTZ

```

Feedbacks globais da plataforma. O administrador pode inserir instruções que são automaticamente injetadas no contexto de todos os agentes em todas as análises. Usado para calibrar comportamento da IA sem alterar código.

### public.audit\_logs

```

id          UUID PK
event       TEXT → tipo do evento (ex: 'analise.created',
'admin.plan_activated')
user_id     UUID → quem realizou a ação
target_id   TEXT → ID do objeto afetado
metadata    JSONB → dados adicionais do evento
ip          TEXT → IP do usuário
user_agent  TEXT → browser/cliente usado
created_at  TIMESTAMPTZ

```

Log central de segurança e rastreabilidade. Registra todas as ações sensíveis do sistema: criação e exclusão de análises, ativação/cancelamento de planos, compartilhamentos, buscas administrativas.

Política RLS: apenas service role acessa (completamente opaco para usuários e gerentes).

## public.rate\_limits

```
key          TEXT PK → identificador único do limite (ex: 'analise:user-uuid')
count        INTEGER → número de requisições na janela atual
window_start TIMESTAMPTZ → início da janela de tempo
```

Controle de taxa atômico via função PostgreSQL `upsert_rate_limit`. Garante que limites funcionem corretamente mesmo com múltiplas instâncias paralelas do servidor em execução simultânea.

### 3.3 Índices de Performance

```
-- Análises por usuário (query mais frequente do dashboard)
idx_analises_user_id          ON analises(user_id)
idx_analises_pipeline_stage  ON analises(pipeline_stage)

-- Assinaturas
idx_subscriptions_user_id      ON subscriptions(user_id)
idx_subscriptions_stripe_customer ON subscriptions(stripe_customer_id)

-- Pipeline e membros
idx_deal_members_analise      ON deal_members(analise_id)
idx_deal_members_user         ON deal_members(user_id)
idx_pipeline_events_analise   ON deal_pipeline_events(analise_id)

-- Versões de output
idx_output_versions_analise_step ON deal_output_versions(analise_id,
step_key, version_num DESC)
idx_output_versions_unique      ON deal_output_versions(analise_id,
step_key, version_num)

-- Audit logs (consultas admin por data e usuário)
audit_logs_created_at_idx      ON audit_logs(created_at DESC)
audit_logs_user_id_idx        ON audit_logs(user_id)
audit_logs_event_idx          ON audit_logs(event)
audit_logs_target_id_idx      ON audit_logs(target_id)

-- Logs de IA por análise
idx_audit_logs_analise        ON deal_step_audit_logs(analise_id, step_key,
ran_at DESC)
idx_attestations_analise      ON deal_attestations(analise_id, step_key)
```

## 4. Hierarquia de Acessos e Permissões

### 4.1 Os Três Roles

O sistema possui três roles de usuário com escopo de acesso claramente definido:

#### ADMIN

Acesso irrestrito a toda a plataforma  
Gerencia escritórios, usuários, planos, agentes  
Visualiza todas as análises de todos os usuários  
Consulta métricas, audit logs, webhooks

#### GERENTE

Acesso ao próprio escritório + toda a equipe  
Vê as análises de todos os assessores do escritório  
Convida assessores para o escritório  
Gerencia perfil e dados do escritório

#### ASSESSOR

Acesso exclusivo às próprias análises  
Cria, processa e exporta análises  
Compartilha análises com links externos  
Não acessa dados de outros usuários

### 4.2 Resolução de Role em Runtime

A cada requisição que requer verificação de permissão, o sistema executa:

```
// lib/get-role.ts
async function getUserContext(): Promise<UserContext | null> {
  // 1. Verifica sessão ativa (Supabase Auth)
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return null

  // 2. Busca role na tabela perfis via service role (bypassa RLS)
  const { data: perfil } = await admin
    .from('perfis')
    .select('role, escritorio_id')
    .eq('user_id', user.id)
    .maybeSingle()

  // 3. Default: assessor (nunca eleva automaticamente)
  const role = perfil?.role ?? 'assessor'
```

```

    return { userId, email, role, escritorioId: perfil?.escritorio_id ??
null }
}

```

**Princípio do mínimo privilégio:** usuários sem perfil cadastrado recebem automaticamente o role mais restrito ( **assessor** ). Elevação de privilégio requer ação explícita do admin.

### 4.3 Escopo de Visibilidade de Análises

```

async function getTeamUserIds(ctx: UserContext): Promise<string[] | null>
{
  // Admin: null = sem filtro, acessa tudo
  if (ctx.role === 'admin') return null

  // Gerente: acessa próprias análises + análises de toda a equipe do
escritório
  if (ctx.role === 'gerente') {
    if (!ctx.escritorioId) return [ctx.userId]
    const membros = await admin.from('perfis')
      .select('user_id').eq('escritorio_id', ctx.escritorioId)
    return [ctx.userId, ...membros.map(m => m.user_id)]
  }

  // Assessor: apenas as próprias análises
  return [ctx.userId]
}

```

### 4.4 Matriz de Permissões Completa

| Ação                              | Assessor  | Gerente      | Admin      |
|-----------------------------------|-----------|--------------|------------|
| Criar análise                     | ✓ própria | ✓ própria    | ✓ qualquer |
| Ver análise                       | ✓ própria | ✓ escritório | ✓ todas    |
| Deletar análise                   | ✓ própria | ✓ própria    | ✓ todas    |
| Exportar análise (PDF/PPTX/Excel) | ✓ própria | ✓ escritório | ✓ todas    |
| Compartilhar análise externamente | ✓ própria | ✓ escritório | ✓          |
| Atestar relatório de IA           | ✓         | ✓            | ✓          |
| Editar perfil do escritório       | ✗         | ✓            | ✓          |
| Convidar assessores               | ✗         | ✓            | ✓          |

|                                      |         |                |   |
|--------------------------------------|---------|----------------|---|
| Adicionar membros ao deal            | ✓ owner | ✓ owner/gestor | ✓ |
| Ver audit logs                       | ✗       | ✗              | ✓ |
| Gerenciar planos de usuários         | ✗       | ✗              | ✓ |
| Gerenciar escritórios                | ✗       | ✗              | ✓ |
| Banir/desbanir usuários              | ✗       | ✗              | ✓ |
| Editar prompts dos agentes           | ✗       | ✗              | ✓ |
| Gerenciar aprendizados globais       | ✗       | ✗              | ✓ |
| Ver métricas da plataforma           | ✗       | ✗              | ✓ |
| Gerenciar aprendizados do escritório | ✗       | ✓              | ✓ |

## 5. Gestão de Escritórios e Usuários

### 5.1 O Modelo Multiescritório

O RR7x Capital Hub é construído sobre um modelo multiempresa onde cada **escritório** é uma unidade organizacional isolada. Um escritório pode ser:

- Um escritório de assessoria financeira autônomo (AAI)
- Uma boutique de M&A
- Uma mesa de uma plataforma de investimentos
- Um escritório família (family office)

Cada escritório possui:

- Identidade visual própria (nome, logo, tagline, CNPJ, contato)
- Plano de assinatura próprio (avulso, recorrente ou enterprise)
- Limite de análises específico (ou ilimitado)
- Aprendizados de IA específicos que se somam aos globais
- Equipe com roles definidos (gerente + assessores)

A separação entre escritórios é total no plano de dados: um assessor de um escritório nunca enxerga dados de outro escritório — nem análises, nem membros, nem configurações.

### 5.2 Criação e Configuração de um Escritório

Via painel admin:

1. Admin acessa `/dashboard/admin/escritorios`
2. Cria um novo escritório com nome (único campo obrigatório)

3. Define o plano e limites via PATCH
4. Convida o gerente do escritório via email

#### Fluxo de convite de gerente:

```
Admin convida gerente (email)
→ Supabase dispara email com link para /auth/definir-senha
→ Usuário define senha e completa cadastro
→ Perfil criado automaticamente com role='gerente' e escritorio_id correto
→ Gerente tem acesso imediato ao escritório e pode convidar assessores
```

#### Convite interno (gerente convida assessor):

```
Gerente acessa /dashboard/equipe
→ Informa email do assessor
→ POST /api/gerente/convidar
→ Sistema valida role do solicitante (deve ser gerente)
→ auth.admin.inviteUserByEmail com metadata de role e escritorio_id
→ Perfil pré-criado com role='assessor' e escritorio_id do gerente
→ Assessor recebe email e define senha
```

### 5.3 White-label Automático

Quando um assessor cria uma análise, o sistema resolve e injeta a identidade do escritório no contexto de cada agente de IA:

```
DADOS DO ESCRITÓRIO / ASSESSORIA:
Nome: [nome do escritório]
CNPJ: [cnpj]
Endereço: [endereço]
...

INSTRUÇÃO: Este documento é emitido em nome do escritório acima – use o nome
"[nome do escritório]" como assessoria responsável, não "RR7x Capital Hub".
Se uma Logo URL estiver disponível, inclua no cabeçalho como ![Logo](url).
Use o email, site e telefone do escritório no rodapé.
```

Resultado: os relatórios gerados são brandados com a identidade do escritório do assessor. O assessor entrega ao cliente um relatório que parece produzido internamente pela assessoria.

### 5.4 Plano por Escritório vs. Plano por Usuário

**Modelo por usuário (padrão):** Cada usuário tem sua própria **subscription**. É o modelo padrão para compras via Stripe. Adequado para assessores individuais ou escritórios onde cada assessor tem sua própria conta.

**Modelo por escritório (enterprise):** O admin configura o plano diretamente no registro do escritório via `PATCH /api/admin/escritorios`. O campo `plano_limite_analises` controla quantas análises o escritório como um todo pode realizar. Gerenciado manualmente pelo admin e não passa pelo Stripe — adequado para contratos enterprise com faturamento via nota fiscal.

5.5 Banimento e Controle de Acesso

O admin pode controlar o acesso de forma granular:

| Ação                    | Efeito                                       | Reversível |
|-------------------------|----------------------------------------------|------------|
| Banir usuário           | Invalida sessões, bloqueia login (~100 anos) | ✓          |
| Desbanir                | Restaura acesso imediatamente                | —          |
| Remover do escritório   | Deleta perfil, preserva conta e análises     | ✓          |
| Excluir permanentemente | Remove perfil + conta Auth                   | ✗          |

Usuários banidos que tentem acessar qualquer endpoint recebem 401 do Supabase antes de qualquer lógica de negócio ser executada.

6. Fluxos Operacionais do Sistema

6.1 Fluxo de Onboarding

```
1. Usuário acessa /auth/login ou recebe convite por email
2. Autenticação via email/senha no Supabase Auth
3. Middleware verifica: nome no user_metadata
   → Se ausente: redireciona para /auth/completar-cadastro
4. Middleware verifica: MFA AAL2
   → Se não configurado: redireciona para /auth/mfa-required
5. Usuário configura TOTP (Google Authenticator, etc.)
6. Valida código TOTP → sessão elevada para AAL2
7. Acesso liberado ao dashboard
```

6.2 Fluxo Completo de Criação de Análise

```
Assessor acessa /dashboard/nova-analise
└─ Preenche Deal Intake:
    └─ Dados do ativo (nome, tipo, estágio, objetivo, ticket, localização)
    └─ Nível de informação disponível
    └─ Informações adicionais e resumo da tese
    └─ Dados opcionais: proprietário (PII), mandato, assessores parceiros
```

- └─ Link de documentos (Google Drive ou URL pública)

Submete → POST /api/analise

- └─ 1. Autenticação verificada (servidor)
- └─ 2. Rate limit verificado (10 análises/hora/usuário)
- └─ 3. Validação Zod de todos os campos
- └─ 4. Assinatura ativa verificada (subscription.status = 'ativo')
- └─ 5. Limite verificado (analises\_restantes > 0 ou NULL)
- └─ 6. Campos PII criptografados com AES-256-GCM
- └─ 7. Análise criada com status='processando'
- └─ 8. Audit log registrado (analise.created)
- └─ 9. analises\_restantes decrementado (se aplicável)
- └─ 10. Retorna { analiseId }

Frontend redireciona para /dashboard/analise/[id]

Pipeline de agentes executa em sequência

### 6.3 Fluxo de Execução de cada Step do Pipeline

POST /api/analise/[id]/step { step: 'pesquisa' }

- └─ Autenticação + verificação de ownership
- └─ Carregamento paralelo (Promise.all):
  - └─ Prompts customizados (agent\_prompts do banco)
  - └─ Dados do escritório (white-label)
  - └─ Feedbacks globais + feedbacks do escritório
  - └─ Dados externos (conforme o step):
    - └─ BCB: SELIC, IPCA, USD/BRL, PIB (pesquisa, estruturacao)
    - └─ CVM: comparáveis, debêntures, CRI, CRA (pesquisa, estruturacao)
    - └─ Documentos do Storage (drive\_intake)
- └─ Construção do contexto:
  - └─ Humanizer Directive (anti-padrão IA)
  - └─ System prompt do agente
  - └─ Feedbacks injetados
  - └─ Deal intake formatado
  - └─ Outputs de steps anteriores (contexto cumulativo)
  - └─ Dados externos reais (BCB, CVM)
  - └─ Bloco do escritório (white-label)
- └─ Execução Anthropic API (streaming)
  - └─ Steps críticos (diagnostico, analise\_ma, estruturacao):
    - └─ Extended Thinking ativo – budget: 8.000 tokens internos
    - └─ max\_tokens: 16.000
  - └─ Demais steps: raciocínio direto, max\_tokens: 10.000



- Streaming token a token → browser via ReadableStream HTTP
- Ao finalizar (finalMessage callback):
  - Output salvo em `analises.outputs[step]`
  - Versão criada em `deal_output_versions`
  - Audit log de IA salvo em `deal_step_audit_logs`

## 6.4 Fluxo de Compartilhamento Externo

1. POST `/api/analise/share { analiseId }`
  - Rate limit: 20 tokens/hora/usuário
  - Token HMAC-SHA256 gerado com TTL 7 dias
  - Link: `/view/[token]`
2. Receptor acessa `/view/[token]` (sem autenticação)
  - Sistema verifica: assinatura HMAC + TTL + não-revogado
  - Se válido: análise exibida em modo somente leitura
  - Acesso registrado: `audit_log(share.accessed, ip, timestamp)`
3. Revogação: DELETE `/api/analise/share { analiseId }`
  - Token registrado como revogado
  - Link existente para de funcionar imediatamente

## 6.5 Fluxo de Pagamento

1. POST `/api/checkout { plano }`
  - Stripe Checkout Session criada
  - metadata: `{ user_id, plano }`
2. Usuário completa pagamento no Stripe (hosted page)
3. Stripe → POST `/api/webhook`
  - Verificação de assinatura (`stripe.webhooks.constructEvent`)
  - Idempotência: `stripe_event_id` verificado
  - Evento registrado ANTES de processar
  - Subscription criada: `analises_restantes = 1 (avulso) | NULL (recorrente)`
4. Usuário retorna para `/dashboard` com assinatura ativa

## 6.6 Fluxo de Atestação Regulatória

1. Assessor revisa output de um agente
2. Clica em "Atestar este relatório"
3. Sistema exibe a declaração legal completa para confirmação

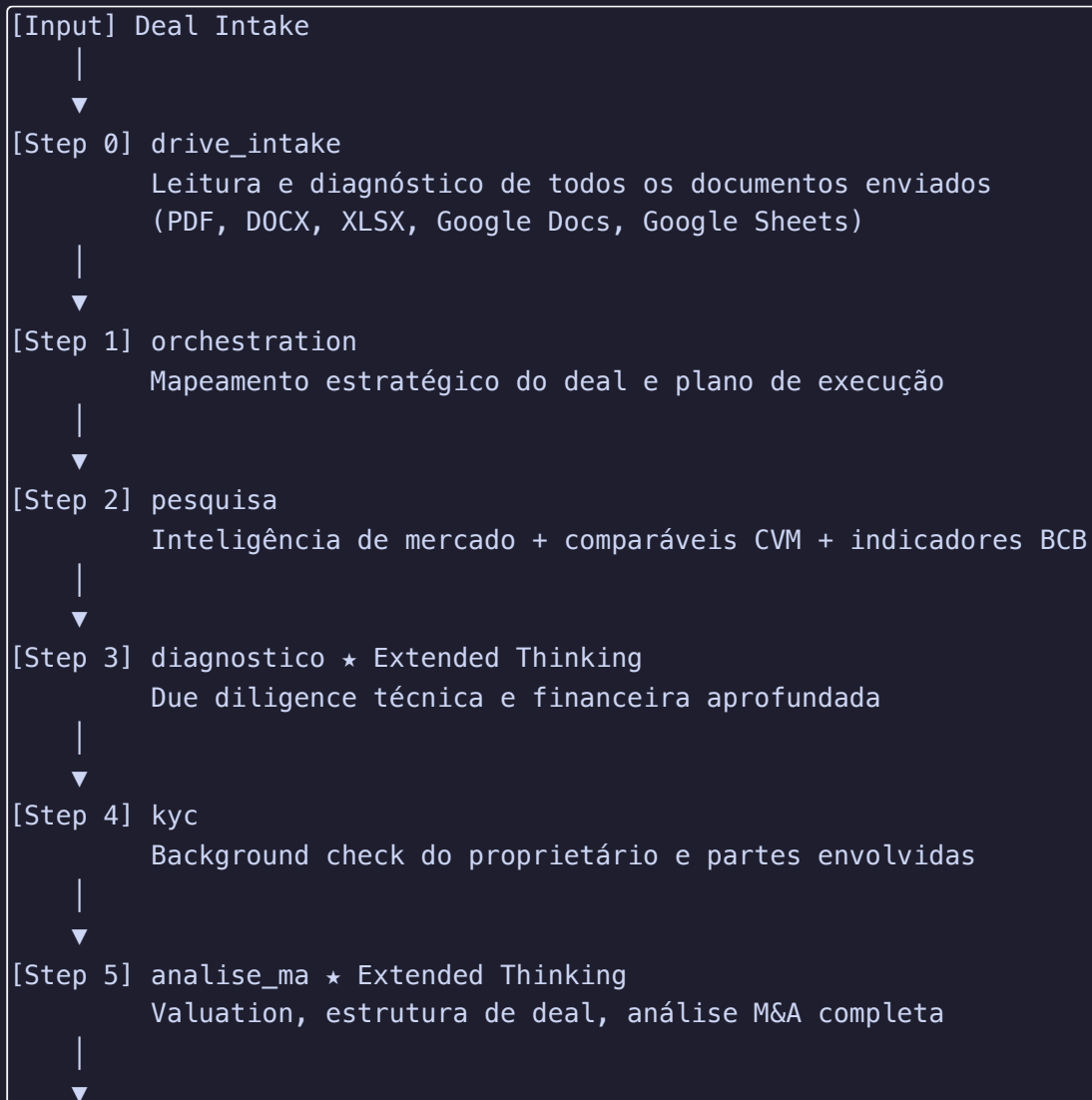
4. Assessor confirma → POST /api/analise/attest
5. IP capturado dos headers (x-forwarded-for / x-real-ip)
6. Atestação salva em deal\_attestations:
  - analise\_id, step\_key, version\_num
  - attested\_by (UUID), attested\_email
  - statement (texto legal completo)
  - ip\_address, attested\_at

Nota: reprocessar o step cria nova versão → nova atestação necessária

## 7. Pipeline de Inteligência — Deal Analysis

### 7.1 Arquitetura do Pipeline

O pipeline é composto por agentes especializados que executam em sequência. Cada agente recebe o intake original mais todos os outputs dos agentes anteriores — raciocínio acumulativo e contextualizado.



```
[Step 6] contratos
        Estrutura jurídica, cláusulas críticas, NDA e minutas
    |
    ▼
[Step 7] originacao
        Estratégia de origemção e mapeamento de compradores
    |
    ▼
[Step 8] estruturacao ★ Extended Thinking
        Estruturação financeira + dados de mercado de capitais (CVM/BCB)
    |
    ▼
[Step 9] maturidade
        Avaliação de maturidade operacional e de governança
    |
    ▼
[Step 10] revisao
        Relatório consolidado e recomendação executiva final
    |
    ▼
[Output] PDF + PPTX + Excel exportáveis | NDA por email
```

★ Extended Thinking ativo: budget de 8.000 tokens de raciocínio interno, max\_tokens de output: 16.000.

## 7.2 Extended Thinking — Raciocínio Profundo

Três steps são classificados como analíticos críticos:

- **diagnostico** : due diligence financeira com números reais
- **analise\_ma** : valuation e estrutura de transação
- **estruturacao** : modelagem financeira e mercado de capitais

Com Extended Thinking ativado, o modelo "pensa" antes de responder — explora hipóteses, reconsidera premissas, identifica inconsistências. O output final é mais preciso, mais fundamentado e mais útil para tomada de decisão de alto valor.

## 7.3 Dados Externos Injetados em Tempo Real

Os agentes não trabalham apenas com os dados fornecidos pelo assessor. O sistema busca dados reais de mercado e os injeta no contexto antes de cada execução:

**Banco Central do Brasil (API aberta — sem cache):**

- Série 12: Meta SELIC (% a.a.)
- Série 433: IPCA variação mensal
- Série 1: Taxa de câmbio USD/BRL (PTAX compra)
- Série 7326: PIB crescimento trimestral

Timeout: 5 segundos. Falha silenciosa — a análise prossegue sem os dados macroeconômicos.

#### **CVM — dados.cvm.gov.br (cache de 24 horas):**

- `cad_cia_aberta.csv` : empresas abertas — comparáveis listados por setor
- `deb_cad.csv` : debêntures cadastradas — curva de crédito corporativo
- `cri_cad.csv` : CRI cadastrados — benchmark de securitização imobiliária
- `cra_cad.csv` : CRA cadastrados — benchmark de securitização do agronegócio

O sistema extrai keywords do setor via `extractSectorKeywords` e filtra comparáveis relevantes automaticamente.

#### **CADE — verificação regulatória automática:**

Verifica se o deal atinge os thresholds de notificação obrigatória (Resolução CADE 33/2022 + Lei 12.529/2011 art. 88):

- Grupo do adquirente: faturamento  $\geq$  R\$ 750MM
- Grupo do vendedor: faturamento  $\geq$  R\$ 75MM

Retorna 4 níveis: `obrigatorio`, `provavel`, `inconclusivo`, `nao_obrigatorio`, com prazo legal e base legal aplicável.

### **7.4 Humanizer Directive — Anti-Padrão de IA**

Todos os agentes recebem como primeiro bloco do system prompt uma diretiva de escrita obrigatória:

#### **Proibido:**

- Palavras-gatilho de IA genérica: "crucial", "fundamental", "vital", "robusto", "abrangente", "ecossistema" (figurado), "jornada" (figurado), "alavancar" (figurado), "sinergias" (sem dado concreto)
- Construções artificiais: "serve como", "está no cerne de", "representa um marco", "destaca-se como"
- Frases de abertura vazias: "Vamos explorar", "Neste relatório, apresentamos"
- Conclusões genéricas: "O futuro é promissor", "Com as ações corretas..."
- Negrito mecânico, emojis em headings, hedging excessivo

#### **Obrigatório:**

- Terminologia técnica financeira preservada: EBITDA, DRS, M&A, CRI, LCI, CRA, SPE, covenant, cap rate, LTV, DSCR, TIR, VPL
- Posição quando os dados sustentam uma conclusão — sem neutralizar artificialmente
- Declarar incertezas com precisão quando existem
- Variar ritmo: frases curtas quando o ponto é simples, mais longas quando a cadeia de raciocínio exige

Resultado: os relatórios soam como documentos escritos por um analista sênior de mercado.

### 7.5 Sistema de Prompts Editáveis

Os prompts de cada agente ficam na tabela `agent_prompts` e são editáveis pelo admin em tempo real via `/dashboard/admin/agentes` :

- Calibrar comportamento de um agente específico sem alterar código
- Adaptar para novos segmentos de mercado sem deploy
- Rollback imediato se uma mudança piorar a qualidade
- O código carrega os prompts do banco a cada execução; se ausentes, usa fallback hardcoded

### 7.6 Aprendizados por Camada

**Aprendizados globais** ( `admin_feedbacks` ): aplicados em todas as análises de todos os escritórios. O admin controla via `/dashboard/admin/aprendizados` .

**Aprendizados por escritório** ( `escritorio_feedbacks` ): aplicados apenas nas análises do escritório específico. O gerente controla via `/dashboard/aprendizados` .

Os dois conjuntos são carregados em paralelo e injetados no contexto com instrução clara: "aplique como contexto complementar, nunca sobreponha às regras do agente."

### 7.7 Exportações Disponíveis

| Formato       | Endpoint                                   | Conteúdo                                                        |
|---------------|--------------------------------------------|-----------------------------------------------------------------|
| PDF           | <code>GET /api/analise/export-pdf</code>   | Documento institucional completo com logo do escritório         |
| PowerPoint    | <code>GET /api/analise/export-pptx</code>  | Apresentação executiva por seção                                |
| Excel         | <code>GET /api/analise/export-excel</code> | Modelo financeiro com múltiplas abas + análise de sensibilidade |
| NDA por email | <code>POST /api/analise/nda-email</code>   | Extraí seção de NDA e envia via Resend para a contraparte       |

## 8. Integrações Externas e Dados de Mercado

### 8.1 Anthropic Claude API

- **Modelo:** `claude-sonnet-4-6`
- **Streaming:** `anthropic.messages.stream()` — output em tempo real token a token
- **Timeout:** `maxDuration = 300` segundos por step (configuração Vercel)

- **Extended Thinking:** `{ type: 'enabled', budget_tokens: 8000 }` para steps críticos
- **Prompt caching:** system prompts com `cache_control: { type: 'ephemeral' }` — reduz custo e latência em execuções repetidas

## 8.2 Supabase

- **Auth:** gerenciamento completo de sessão, MFA (TOTP), convites por email, banimentos
- **PostgreSQL:** banco relacional com RLS nativo em todas as tabelas
- **Storage:** buckets para documentos de análise ( `analises/` , privado) e logos ( `logos/` , público)
- **RPC:** função `upsert_rate_limit` para operações atômicas de rate limiting
- **SSR:** cookies de sessão gerenciados via `@supabase/ssr`

## 8.3 Stripe

- **Checkout:** página hosted do Stripe (PCI DSS Compliant — dados de cartão nunca passam pelo servidor)
- **Modos:** `payment` (avulso) e `subscription` (recorrente)
- **Webhooks:** `checkout.session.completed` , `customer.subscription.deleted`
- **Idempotência:** todos os eventos registrados antes de processar
- **Metadados:** `user_id` e `plano` passados como metadata na sessão e validados no webhook

## 8.4 Resend

- **Emails transacionais:** conclusão de análise (para o assessor), erro no pipeline (para o admin), NDA para contraparte
- **Template:** HTML responsivo com cores da marca e link direto para a análise

## 8.5 Banco Central do Brasil

- **Endpoint:** `api.bcb.gov.br/dados/serie/bcdata.sgs.[codigo]/dados/ultimos/1?formato=json`
- **Timeout:** 5 segundos por série
- **Cache:** desabilitado ( `cache: 'no-store'` ) — dados sempre atualizados
- **Falha:** silenciosa, não bloqueia o pipeline

## 8.6 CVM — Dados Abertos

- **Fonte:** `dados.cvm.gov.br` (CSVs públicos)
- **Cache:** 24 horas via Next.js Data Cache ( `next: { revalidate: 86400 }` )
- **Encoding:** arquivos em latin-1, decodificados corretamente via `TextDecoder('latin-1')`
- **Timeout:** 14 segundos (CSVs volumosos)

## 8.7 Google Drive e Documentos

- **Google Docs públicos:** export direto via `docs.google.com/document/d/[id]/export?format=txt`
  - **Google Sheets públicos:** export via `export?format=csv`
  - **Outros links:** lidos via Jina AI reader ( `r.jina.ai/[url]` )
  - **Validação prévia:** `GET /api/validar-drive` verifica acessibilidade antes de aceitar a URL no intake
- 

## 9. Modelo de Negócio e Planos de Assinatura

---

### 9.1 Planos Disponíveis

#### Avulso

- 1 análise completa
- Cobrado via Stripe (pagamento único)
- `analises_restantes = 1` , decrementado após uso
- Sem renovação automática

#### Recorrente

- Análises ilimitadas
- Cobrado via Stripe (assinatura mensal)
- `analises_restantes = NULL` (ilimitado)
- Renovação mensal automática; cancelamento encerra no fim do período pago

#### Enterprise

- Solicitação via email ( `gestor@renanregonato.com.br` )
- Plano customizado, faturamento via nota fiscal
- Gerenciado manualmente pelo admin no painel
- `plano_limite_analises` configurável por escritório
- Sem dependência de Stripe

## 9.2 Lógica de Controle de Acesso por Plano

Criar análise → verificar subscription ativa:

1. Existe subscription com status='ativo'?  
→ Não: HTTP 403 – "Sem assinatura ativa. Adquira um plano para continuar."
2. analyses\_restantes == NULL?  
→ Sim: ilimitado, prossegue
3. analyses\_restantes <= 0?  
→ Sim: HTTP 403 – "Limite de análises atingido."
4. analyses\_restantes > 0?  
→ Prossegue + decrementa após criação bem-sucedida

## 9.3 Gestão Administrativa de Planos

O admin pode ativar, alterar ou cancelar planos de qualquer usuário sem depender do Stripe:

- `POST /api/admin { plano, user_id }` → ativa ou atualiza
- `DELETE /api/admin { user_id }` → cancela assinatura

Útil para: trials gratuitos, resolução de chargebacks, upgrades manuais, contas de teste.

# 10. Segurança, Autenticação e Autorização

## 10.1 Camadas de Autenticação (6 camadas independentes)

### Camada 1 — Middleware Edge (middleware.ts)

Executado antes de qualquer Route Handler. Verifica sessão JWT via `supabase.auth.getUser()`.

Se não autenticado em rotas `/dashboard/*`, redireciona para login. Não pode ser bypassado — é o primeiro ponto de entrada.

### Camada 2 — Gate de Onboarding

Verifica `user_metadata.nome`. Usuário sem nome não acessa o dashboard — é redirecionado para completar cadastro. Previne contas em estado inconsistente.

### Camada 3 — 2FA Obrigatório (AAL2)

```
const { data: aal } = await
supabase.auth.mfa.getAuthenticatorAssuranceLevel()
```



```
if (aal.currentLevel !== 'aal2') {  
  redirect('/auth/mfa-required?next=' + pathname)  
}
```

**aal2** significa que o usuário passou pelo segundo fator **nesta sessão** — não apenas que MFA está configurado. Um login válido sem MFA ainda bloqueia o dashboard.

#### Camada 4 — Verificação por Route Handler

Cada Route Handler protegido repete `supabase.auth.getUser()` no servidor. Não confia no estado do cliente.

#### Camada 5 — Verificação de Role

Rotas administrativas chamam `getUserContext()` e verificam `ctx.role === 'admin'`. Um assessor que conheça a URL de uma rota admin recebe HTTP 403 imediatamente.

#### Camada 6 — Row Level Security (PostgreSQL)

Mesmo que um bug na aplicação entregasse um cliente Supabase SSR incorreto, o RLS garante que queries retornem apenas os dados do usuário autenticado. É a rede de segurança de banco de dados, independente da lógica da aplicação.

### 10.2 Proteção Contra Elevação de Privilégio

O sistema nunca aceita role do cliente. A sequência é sempre:

1. Cliente faz request
2. Servidor extrai `user_id` do JWT assinado pelo Supabase
3. Servidor busca `role` na tabela `perfis` via service role
4. Role determinada server-side é usada para autorização

Não existe endpoint que aceite role do corpo da requisição para autorização.

### 10.3 Validação de Inputs com Zod

Todos os endpoints que recebem dados do cliente validam o body com schemas Zod:

- Tipos corretos (string, número, UUID, email, URL)
- Tamanhos máximos (previne inputs gigantes que causariam OOM ou injeção)
- Formatos válidos (email válido, URL válida, UUID bem formado)
- Campos proibidos ( `.strict()` no schema do escritório rejeita campos não declarados)

Qualquer falha retorna HTTP 400 com detalhes dos campos inválidos. Nunca executa com dados parcialmente válidos.

### 10.4 Rate Limiting Distribuído

Implementado via função PostgreSQL atômica que usa `FOR UPDATE` para evitar race conditions entre instâncias paralelas:

| Endpoint                             | Limite                       |
|--------------------------------------|------------------------------|
| <code>POST /api/analise</code>       | 10 análises / hora / usuário |
| <code>POST /api/analise/share</code> | 20 tokens / hora / usuário   |

Quando o limite é atingido: HTTP 429 com header `Retry-After: [segundos]`.

**Fail open por design:** se o Supabase estiver indisponível, o rate limit falha silenciosamente e a requisição é permitida — disponibilidade tem prioridade sobre rate limiting em falhas de infraestrutura.

## 11. Proteção de Dados e Criptografia

### 11.1 Criptografia de PII em Repouso (AES-256-GCM)

Campos com dados pessoais identificáveis do proprietário do ativo são criptografados antes de salvar no banco:

```
Campos criptografados:
nomeProprietario      → Nome completo
cpfCnpjProprietario   → CPF ou CNPJ
telefoneProprietario  → Telefone
emailProprietario     → Email
obsProprietario       → Observações sobre o proprietário
```

**Algoritmo:** AES-256-GCM

- Chave: 256 bits (32 bytes via `ENCRYPTION_KEY`)
- IV: 96 bits aleatórios por criptografia — nunca reutilizados
- Auth Tag: 128 bits — garante integridade; adulteração causa erro na deciptação
- Formato no banco: `base64(iv):base64(authTag):base64(ciphertext)`

**Fluxo:** `encryptSensitiveFields(intake)` antes do INSERT →

`decryptSensitiveFields(intake)` antes de enviar ao cliente. Se o dado for adulterado, o campo retorna `[dado inválido]` em vez de lançar exceção.

### 11.2 Tokens de Compartilhamento (HMAC-SHA256)

```
Token = base64url(analiseId:exp:HMAC-SHA256(analiseId:exp, SECRET))
```

- **Secret:** `SHARE_TOKEN_SECRET` (variável de ambiente server-only)
- **TTL:** 7 dias
- **Falsificação:** impossível sem conhecer o secret
- **Revogação:** tokens registrados em `revoked_share_tokens`

11.3 Secrets e Variáveis de Ambiente

| Variável                                   | Uso                        | Exposição ao Browser |
|--------------------------------------------|----------------------------|----------------------|
| <code>ANTHROPIC_API_KEY</code>             | Chave da API Claude        | ✗                    |
| <code>ENCRYPTION_KEY</code>                | Chave AES-256 para PII     | ✗                    |
| <code>SHARE_TOKEN_SECRET</code>            | HMAC para share tokens     | ✗                    |
| <code>STRIPE_SECRET_KEY</code>             | Chave secreta Stripe       | ✗                    |
| <code>STRIPE_WEBHOOK_SECRET</code>         | Verificação de webhook     | ✗                    |
| <code>SUPABASE_SERVICE_ROLE_KEY</code>     | Service role (bypassa RLS) | ✗                    |
| <code>RESEND_API_KEY</code>                | Email transacional         | ✗                    |
| <code>NEXT_PUBLIC_SUPABASE_URL</code>      | URL do Supabase            | ✓ (por design)       |
| <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> | Chave anônima              | ✓ (restrito por RLS) |

As variáveis `NEXT_PUBLIC_*` são públicas por design — a segurança está no RLS do banco, não na obscuridade da chave anônima.

12. Auditoria, Rastreabilidade e Conformidade

12.1 Dois Níveis de Audit Log

Nível 1 — Audit Log de Segurança ( `audit_logs` ):

| Evento                       | Gatilho                            |
|------------------------------|------------------------------------|
| <code>analise.created</code> | Análise criada com sucesso         |
| <code>analise.deleted</code> | Análise excluída                   |
| <code>share.created</code>   | Link de compartilhamento gerado    |
| <code>share.revoked</code>   | Link revogado                      |
| <code>share.accessed</code>  | Link acessado por receptor externo |

|                                   |                                   |
|-----------------------------------|-----------------------------------|
| <code>admin.plan_activated</code> | Admin ativou plano                |
| <code>admin.plan_cancelled</code> | Admin cancelou assinatura         |
| <code>admin.user_searched</code>  | Admin buscou usuário por email    |
| <code>invite.sent</code>          | Convite enviado para novo usuário |

Cada evento registra: `user_id` , `target_id` , `metadata` , `ip` , `user_agent` , `timestamp` .

## Nível 2 — Audit Log de IA ( `deal_step_audit_logs` ):

Registra cada execução de agente com:

- Modelo utilizado e versão exata
- Tokens consumidos (input + output)
- Snapshot exato do intake enviado para a IA
- Steps anteriores presentes no contexto
- Dados externos injetados (BCB, CVM, documentos)
- Tempo de execução em milissegundos

Isso permite responder: *"O que exatamente a IA recebeu como input quando produziu esta conclusão?"* — essencial para disputas e revisões regulatórias.

## 12.2 Conformidade com ICVM 598/2018

A ICVM 598/2018 regula a atividade de consultoria de valores mobiliários e exige que recomendações sejam fundamentadas, rastreáveis e de responsabilidade do profissional habilitado.

O sistema implementa conformidade estrutural:

1. **Audit log de IA:** documenta exatamente o que a IA recebeu e produziu
2. **Versionamento de outputs:** preserva histórico completo de versões
3. **Atestação digital:** assessor assina declaração formal de supervisão humana
4. **IP e timestamp:** prova forense de quando e de onde ocorreu a revisão
5. **Rastreabilidade de dados externos:** o log registra quais dados de mercado foram injetados

A declaração de atestação inclui explicitamente "conforme ICVM 598/2018 e Código ANBIMA para M&A."

# 13. Estratégias Anti-Fraude e Prevenção de Vazamentos

## 13.1 Proteção Contra Enumeração de Dados

**Opacidade de erros:** Endpoints que verificam ownership retornam HTTP 404 (não encontrado) em vez de HTTP 403 (proibido) quando um usuário tenta acessar uma análise de outro. O atacante não consegue confirmar se um ID existe.

**UUID v4 para IDs:** Todos os IDs são UUID v4 aleatórios — impossíveis de enumerar sequencialmente. Não existe `/api/analise/1` , `/api/analise/2` .

**RLS como rede de segurança:** Mesmo que um bug na aplicação use o cliente SSR errado, o banco só retorna dados do usuário autenticado.

## 13.2 Proteção do Pipeline de IA

**Rate limiting por usuário:** Previne que um único usuário esgote a quota da API Anthropic ou crie análises em massa para fins abusivos.

**Verificação de assinatura antes da IA:** O sistema verifica subscription ativa antes de iniciar qualquer processamento. Sem assinatura válida, nenhuma chamada à Anthropic API é realizada.

**Streaming sem buffer completo:** O output é transmitido diretamente do Anthropic para o browser. Não existe buffer completo do output na memória do servidor em nenhum momento — reduz exposição de dados em caso de falha.

## 13.3 Proteção do Webhook Stripe

**Verificação de assinatura:** `stripe.webhooks.constructEvent(body, sig, secret)` — qualquer request sem assinatura válida retorna HTTP 400 imediatamente.

**Idempotência:** O `stripe_event_id` é registrado antes de processar o evento. Segunda tentativa do mesmo evento é ignorada silenciosamente.

**Validação de metadata:** `user_id` validado como UUID v4 (regex), `plano` validado como enum. Metadata inválida retorna HTTP 400 antes de qualquer operação no banco.

## 13.4 Proteção de Links de Compartilhamento

- Tokens impossíveis de falsificar (HMAC-SHA256 com secret server-side)
- TTL de 7 dias — links expiram automaticamente
- Revogação manual disponível a qualquer momento
- Rate limit de 20 tokens/hora por usuário
- Acesso registrado em audit log (IP, timestamp, user agent do receptor)

---

## 14. LGPD e Privacidade

---

## 14.1 Dados Pessoais no Sistema

### Dados dos usuários da plataforma:

- Email (identificação única)
- Nome (user\_metadata)
- IP de acesso (audit logs de segurança)
- User-agent (audit logs de segurança)

### Dados de terceiros (proprietários de ativos):

- Nome completo (criptografado em repouso)
- CPF/CNPJ (criptografado em repouso)
- Telefone (criptografado em repouso)
- Email (criptografado em repouso)
- Observações do assessor sobre o proprietário (criptografado em repouso)

## 14.2 Medidas Técnicas de Proteção

### Dados dos usuários:

- Sessão via JWT com expiração natural
- Senhas gerenciadas pelo Supabase Auth (nunca armazenadas em plaintext)
- MFA obrigatório para acesso ao dashboard
- Audit logs de IP para rastreabilidade, não para perfilamento

### Dados de terceiros:

- AES-256-GCM em repouso — aparecem no banco como ciphertext opaco
- Decriptação apenas em memória do servidor, nunca em logs ou caches
- RLS garante que apenas o owner da análise e o admin acessam dados decriptados
- Compartilhamento externo via share token não expõe PII do proprietário

## 14.3 Base Legal para Processamento

- **Usuários da plataforma:** execução de contrato (uso do serviço)
- **Dados de proprietários de ativos:** legítimo interesse do assessor na condução de due diligence, conforme regulação do mercado de capitais

## 14.4 Direitos dos Titulares

- **Exclusão de análise:** `DELETE /api/analise/[id]` — remove análise e arquivos do Storage
- **Exclusão de conta:** admin pode excluir usuário permanentemente via painel

- **Portabilidade:** exportação em PDF, PPTX e Excel disponível a qualquer momento

---

## 15. Escalabilidade, Performance e Redundância

---

### 15.1 Escalabilidade Horizontal

**Vercel Edge Network:** O servidor Next.js é serverless — não existe servidor fixo. Cada request é servido por uma instância efêmera que escala automaticamente. Centenas de requests simultâneos são absorvidos sem intervenção operacional.

**Supabase:** PostgreSQL gerenciado com PgBouncer (connection pooling). Suporta escalonamento vertical e réplicas de leitura sem mudança no código da aplicação.

**Estado stateless:** O servidor não mantém estado entre requests. Toda informação de sessão está no JWT (Supabase) ou no banco. Qualquer instância pode servir qualquer request.

### 15.2 Performance de Queries

**Índices estratégicos:** Todas as queries frequentes de produção têm índices correspondentes. A query mais crítica — buscar análises de um usuário — tem `idx_analises_user_id`.

**Queries paralelas:** Operações independentes são executadas com `Promise.all`:

```
const [prompts, escritorioData, bcbData, cvmCapital, cvmComps] = await
Promise.all([
  loadPrompts(),
  loadEscritorio(analise.user_id),
  fetchBCBIndicators(),
  fetchCapitalMarketsData(sectorKeywords),
  fetchListedComparables(sectorKeywords),
])
```

**Cache de dados CVM:** Arquivos CSV da CVM têm cache de 24 horas via Next.js Data Cache. A primeira request do dia baixa e processa o arquivo; as seguintes são servidas do cache.

**Prompt caching Anthropic:** System prompts com `cache_control: { type: 'ephemeral' }` — o Anthropic mantém esses blocos por 5 minutos, reduzindo custo e latência em execuções sequenciais.

### 15.3 Tratamento de Falhas

**APIs externas (BCB, CVM):** Timeout agressivo e fallback silencioso. A análise prossegue sem os dados se as APIs estiverem indisponíveis.

**Falhas de IA:** Erros na stream da Anthropic são capturados e enviados ao cliente com prefixo

`\x00ERROR:` . O frontend detecta o marcador e exibe mensagem de erro sem deixar o usuário preso em "processando". O step pode ser reprocessado individualmente.

**Falhas de audit log:** `audit()` nunca lança exceção — erros são logados no console mas não interrompem o fluxo principal.

**Idempotência de webhooks:** Garante que falhas de rede e retries do Stripe não causem duplicações de subscriptions.

## 15.4 Proteção Contra Cargas Pesadas

- `maxDuration: 300` — rotas de IA têm timeout de 5 minutos no Vercel
- `max_tokens` limitado por step (10.000 ou 16.000) — previne respostas infinitas
- Rate limiting por usuário — previne múltiplas análises simultâneas do mesmo usuário

## 15.5 Estrutura para Crescimento

O modelo de dados é agnóstico ao número de escritórios. Adicionar 1.000 escritórios é apenas inserção de registros nas tabelas existentes — nenhuma mudança estrutural.

O pipeline de IA é extensível: novos steps são adicionados incluindo um case no `getStepArgs()` e um registro em `agent_prompts` . Nenhuma mudança arquitetural necessária.

---

## 16. Cenários de Uso Reais

---

### Cenário 1: Boutique de M&A — Deal de Empresa Familiar

**Contexto:** Um escritório de M&A recebe mandato sell-side de uma empresa familiar de manufatura, ticket de R\$ 80MM.

#### Fluxo operacional:

1. Admin cria o escritório "M&A Partners" e convida João (gerente)
2. João convida Maria (assessora) via `/dashboard/equipe`
3. Maria preenche o intake com DRE dos últimos 3 anos via link do Google Drive
4. Pipeline executa: lê os DREs → pesquisa comparáveis do setor → diagnóstico financeiro (Extended Thinking) → análise M&A com valuation (Extended Thinking) → contratos e NDA → originação de compradores → estruturação → maturidade → relatório final
5. João, como gerente, vê a análise no dashboard do escritório
6. Maria atesta cada relatório conforme ICVM 598/2018
7. Exporta pitch book em PPTX para apresentar ao comprador



**Resultado:** 45 minutos de processamento vs. 10 dias de trabalho manual. Relatório brandado com logo e dados do escritório "M&A Partners", não com a marca RR7x.

## Cenário 2: Plataforma de Investimentos — Análise de CRI

**Contexto:** Uma plataforma precisa analisar um CRI de R\$ 200MM lastreado em recebíveis de shopping centers.

### Fluxo:

1. Admin ativa plano recorrente para o escritório da plataforma (análises ilimitadas)
2. Assessor cria análise com tipo "Estruturado — CRI"
3. Step **estruturacao** recebe automaticamente:
  - Dados de CRI cadastrados na CVM (benchmark)
  - Taxa SELIC e IPCA do BCB (referência para spread)
  - Extended Thinking para modelagem completa da estrutura
4. Relatório recomenda taxa, prazo, subordinação e covenants
5. Excel exportado serve como base para o modelo financeiro

## Cenário 3: Assessor Individual — Deal de FII

**Contexto:** Assessor autônomo (AAI) analisa um FII de logística para recomendação a cliente.

### Fluxo:

1. Acessa **/dashboard/planos**, seleciona Avulso, paga via Stripe
2. Webhook ativa subscription com **analises\_restantes = 1**
3. Cria análise → pipeline executa → **analises\_restantes** vai para 0
4. Tenta criar nova análise → HTTP 403 "Limite de análises atingido"
5. Precisa adquirir novo plano ou upgrade para recorrente

## Cenário 4: Compartilhamento com Investidor Externo

**Contexto:** Assessor quer compartilhar due diligence com investidor que não tem conta na plataforma.

### Fluxo:

1. Clica em "Compartilhar" → token HMAC gerado → link copiado
2. Investidor acessa o link sem fazer login
3. Sistema valida HMAC + TTL (7 dias) → análise exibida em modo leitura
4. Audit log registra: **share.accessed** com IP e timestamp do investidor
5. Após conclusão da negociação, assessor revoga o link

## Cenário 5: Deal de R\$ 1,2B — Compliance com CADE

**Contexto:** Deal de grande porte onde compliance precisa verificar obrigação de notificação ao CADE.

**Fluxo:**

1. Step `analise_ma` detecta automaticamente: faturamento do adquirente > R\$ 750MM
2. Resultado no relatório: `status: 'obrigatorio'` — notificação obrigatória ao CADE
3. Base legal: Lei 12.529/2011 art. 88 + Resolução CADE 33/2022
4. Prazo legal de submissão informado
5. Membro de compliance adicionado ao deal com role `compliance`
6. Membro revisa, registra aprovação → deal avança de `compliance` para `comite`

## Cenário 6: Envio de NDA para Contraparte

**Contexto:** Após análise de contratos, assessor precisa enviar o NDA para o vendedor assinar.

**Fluxo:**

1. Pipeline concluiu step `contratos` com NDA estruturado
2. Assessor clica em "Enviar NDA por email"
3. `POST /api/analise/nda-email` → sistema extrai a seção de NDA do relatório de contratos automaticamente
4. Email enviado via Resend para o endereço do vendedor com o NDA formatado
5. Assessor recebe confirmação de envio

---

## 17. Limitações Estruturais e Proteções do Sistema

### 17.1 Limitações Intencionais

**Uma subscription por usuário:** O sistema não suporta múltiplas subscriptions ativas simultâneas. A ativação de um novo plano substitui o existente (upsert). Isso simplifica a gestão e previne duplicidades.

**Um escritório por usuário:** Um usuário só pode estar vinculado a um escritório via `perfis.escriptorio_id`. Para operar em múltiplos escritórios, o assessor precisaria de contas separadas.

**Pipeline sequencial:** Os steps executam em sequência — cada agente depende do output do anterior. Não existe execução paralela de steps dentro de uma mesma análise.

**Timeout de 5 minutos por step:** Análises de ativos com documentação muito volumosa podem atingir o limite. O sistema detecta via `\x00ERROR:` na stream e permite reprocessar o step específico sem repetir os

anteriores.

## 17.2 Proteções Estruturais (Invariantes do Sistema)

**Impossível criar análise sem assinatura:** A verificação de subscription acontece antes de qualquer chamada à API da Anthropic. Não existe caminho de código que crie análise sem subscription válida.

**Impossível elevar role via API:** Não existe endpoint que aceite role do cliente para autorização. O único endpoint que muda roles é `POST /api/admin/usuarios`, que requer `role='admin'` para ser acessado.

**Impossível acessar análise de outro usuário (assessor):** O RLS do banco e a verificação de `analise.user_id === user.id` no Route Handler são barreiras independentes — uma não depende da outra.

**Impossível forjar share token:** HMAC-SHA256 com secret armazenado apenas server-side. Qualquer alteração no token invalida a assinatura.

**Impossível reprocessar step sem ownership:** A verificação `analise.user_id !== user.id` && `user.email !== ADMIN_EMAIL` no step handler garante que apenas o owner ou admin possam reprocessar.

---

## 18. Visão Estratégica — Produto SaaS Escalável

---

### 18.1 Posicionamento de Mercado

O RR7x Capital Hub ocupa um segmento específico e defensável: automação de inteligência para M&A e operações estruturadas no mercado financeiro brasileiro, com conformidade regulatória nativa.

As barreiras de entrada são compostas por:

1. **Dados de mercado integrados** (BCB + CVM + CADE): não é só um wrapper de IA
2. **Conformidade regulatória estrutural:** atestação digital, audit log de IA, ICVM 598/2018
3. **White-label multiescritório:** cada escritório opera como se a plataforma fosse sua
4. **Pipeline especializado:** 11 agentes com conhecimento profundo do mercado financeiro brasileiro
5. **Humanizer Directive:** relatórios que passam pelo crivo do mercado sem soar como IA

### 18.2 Modelo de Crescimento

**Expansão vertical (mais valor por escritório):**

- Novos tipos de análise (FIAGRO, FIP, debêntures, operações de securitização)
- Integração com bureaux de crédito para KYC estruturado
- Modelo financeiro automatizado (DCF, LBO, comparáveis de mercado)

- Assinatura digital de contratos gerados (DocuSign, Clicksign)

#### **Expansão horizontal (mais escritórios):**

- O modelo multiescritório é infinitamente escalável sem mudança estrutural
- Cada novo escritório é apenas um registro no banco + configuração de plano
- Parcerias com plataformas de investimento (oferecer como white-label B2B2B)

#### **Expansão de inteligência:**

- Aprendizado contínuo com feedbacks por escritório
- Benchmarking anônimo entre deals similares
- Alertas proativos quando comparáveis relevantes são registrados na CVM

### **18.3 Direções de Roadmap**

#### **Curto prazo:**

- Portal de investidores com conta própria (não apenas link anônimo)
- Notificações em tempo real via Supabase Realtime durante o pipeline
- Dashboard de métricas por escritório (análises/mês, taxa de aprovação, ticket médio)

#### **Médio prazo:**

- API pública para integração em sistemas externos dos escritórios
- Módulo de CRM de deals (pipeline de vendas integrado à análise)
- Relatório de portfólio para gestores com múltiplos ativos em análise simultânea

#### **Longo prazo:**

- Marketplace anônimo de deals (conectar compradores e vendedores sem revelar as partes)
- Módulo de submissão ao CADE (draft automatizado da petição de ato de concentração)
- Certificação de conformidade exportável para comitês de investimento

### **18.4 Padrões Técnicos para Crescimento Sustentável**

**Migrations controladas:** Schema SQL versionado ( `001_` , `002_` , `003_` , `004_` ) executado manualmente no Supabase. Mudanças de schema são revisadas, documentadas e aplicadas de forma controlada.

**Separação de concerns:** Toda lógica de negócio fica em `lib/` . Route Handlers são finos — recebem, validam e delegam. Testável independentemente da camada HTTP.

**Tipos centralizados:** `lib/types.ts` e `lib/schemas.ts` são a fonte de verdade. Mudanças no contrato da API começam aqui.

**Prompts como dados:** Os prompts dos agentes vivem no banco, não no código. Iteração rápida sem deploy. O código só precisa de deploy quando a lógica de orquestração muda.

**Fail-safe por design:** Falhas em sistemas auxiliares (audit log, email, APIs externas) nunca interrompem o fluxo principal. O sistema degrada graciosamente.

## 18.5 Métricas de Saúde da Plataforma

O painel admin em `/dashboard/admin` fornece em tempo real:

- Total de clientes cadastrados
- Assinaturas ativas por plano (avulso, recorrente, enterprise)
- Total de análises e análises criadas no dia
- Distribuição por status (concluído, processando, erro)
- Gráfico de análises por dia (últimos 14 dias)

## Apêndice A — Variáveis de Ambiente

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Anthropic
ANTHROPIC_API_KEY=

# Stripe
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
STRIPE_PRICE_AVULSO=
STRIPE_PRICE_RECURRENTE=

# Resend
RESEND_API_KEY=
RESEND_FROM_EMAIL=

# Segurança
ENCRYPTION_KEY=           # 64 hex chars = 32 bytes AES-256
SHARE_TOKEN_SECRET=       # string aleatória >= 32 chars

# URLs
```

NEXT\_PUBLIC\_SITE\_URL=https://rr7x-portal.vercel.app  
NEXT\_PUBLIC\_APP\_URL=https://rr7x-portal.vercel.app

## Apêndice B — Registro de Eventos de Audit Log

| Evento               | Tabela               | Gatilho                     |
|----------------------|----------------------|-----------------------------|
| analise.created      | audit_logs           | POST /api/analise           |
| analise.deleted      | audit_logs           | DELETE /api/analise/[id]    |
| share.created        | audit_logs           | POST /api/analise/share     |
| share.revoked        | audit_logs           | DELETE /api/analise/share   |
| share.accessed       | audit_logs           | GET /view/[token]           |
| admin.plan_activated | audit_logs           | POST /api/admin             |
| admin.plan_cancelled | audit_logs           | DELETE /api/admin           |
| admin.user_searched  | audit_logs           | GET /api/admin?email=       |
| invite.sent          | audit_logs           | POST /api/gerente/convidar  |
| Execução de IA       | deal_step_audit_logs | POST /api/analise/[id]/step |
| Atestação            | deal_attestations    | POST /api/analise/attest    |
| Evento de pipeline   | deal_pipeline_events | POST /api/analise/pipeline  |

## Apêndice C — Glossário

| Termo             | Definição no contexto do sistema                                                          |
|-------------------|-------------------------------------------------------------------------------------------|
| Deal Intake       | Formulário estruturado de entrada de dados de um ativo para análise                       |
| Step              | Cada agente de IA no pipeline (drive_intake, pesquisa, diagnostico, etc.)                 |
| Output            | Texto gerado por um agente para um step específico                                        |
| Extended Thinking | Modo do Claude onde o modelo raciocina internamente antes de responder (budget de tokens) |
| Escritório        | Unidade organizacional multiempresa — assessoria, boutique ou family office               |
| Gerente           | Role que administra um escritório e sua equipe de assessores                              |
| Assessor          | Role padrão — cria e gerencia as próprias análises                                        |
| AAL2              | Authentication Assurance Level 2 — sessão com MFA validado nesta sessão                   |

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <b>RLS</b>                 | Row Level Security — política de segurança no nível do banco de dados                              |
| <b>Service Role</b>        | Cliente Supabase com permissão irrestrita (bypassa RLS) — usado apenas server-side                 |
| <b>Humanizer Directive</b> | Conjunto de regras que impedem outputs com padrões de IA genérica                                  |
| <b>Atestação</b>           | Declaração digital do assessor assumindo responsabilidade pelo relatório de IA                     |
| <b>Pipeline Stage</b>      | Estágio do deal no processo de aprovação (originacao → comite → aprovado)                          |
| <b>Share Token</b>         | Token HMAC que permite acesso externo a uma análise sem autenticação                               |
| <b>White-label</b>         | Relatórios gerados com identidade visual e dados do escritório do assessor                         |
| <b>Fail open</b>           | Comportamento de permitir a operação quando o sistema auxiliar falha                               |
| <b>Idempotência</b>        | Propriedade de uma operação que produz o mesmo resultado independente de quantas vezes é executada |

*Documento produzido em Maio de 2026.*

*Confidencial — destinado à equipe técnica e de gestão da plataforma RR7x Capital Hub.*